

# 1785345131500\_Data\_Visualization\_Seaborn

July 29, 2026

#

## DATA VISUALIZATION WITH SEABORN

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and plots, visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data. In Python, Seaborn is one of the most widely-used libraries for creating attractive, statistical visualizations with minimal code.

Let us dive deeper into Seaborn, its core structures, and its plotting operations.

### 0.1 SEABORN

Seaborn was first released in 2012 by Michael Waskom. It is a statistical visualization library built directly on top of Matplotlib, designed specifically to work seamlessly with **Pandas** DataFrames. A Seaborn visualization fundamentally relies on:

1. **DataFrame-first API:** Most Seaborn functions take a `data=` argument along with column names for `x`, `y`, and `hue`, instead of raw arrays.
2. **Figure-level vs Axes-level functions:** Axes-level functions (like `sns.barplot`) draw onto a single Matplotlib Axes, while Figure-level functions (like `sns.relplot`) can create entire grids of subplots automatically.
3. **Themes:** Built-in styling presets (`darkgrid`, `whitegrid`, `dark`, `white`, `ticks`) that instantly improve the look of any chart.
4. **Palettes:** Curated color palettes designed for both aesthetics and accessibility (e.g., colorblind-safe palettes).

#### 0.1.1 DataFrame-first API

Instead of passing raw lists like Matplotlib, Seaborn lets you simply reference column names, and it handles grouping, coloring, and aggregation automatically.

#### 0.1.2 Themes & Palettes

Calling `sns.set_theme()` once instantly upgrades the visual style of every chart drawn afterward.

```
[1]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
print("Seaborn version:", sns.__version__)
print("Pandas version:", pd.__version__)
```

Seaborn version: 0.13.2

Pandas version: 3.0.2

#

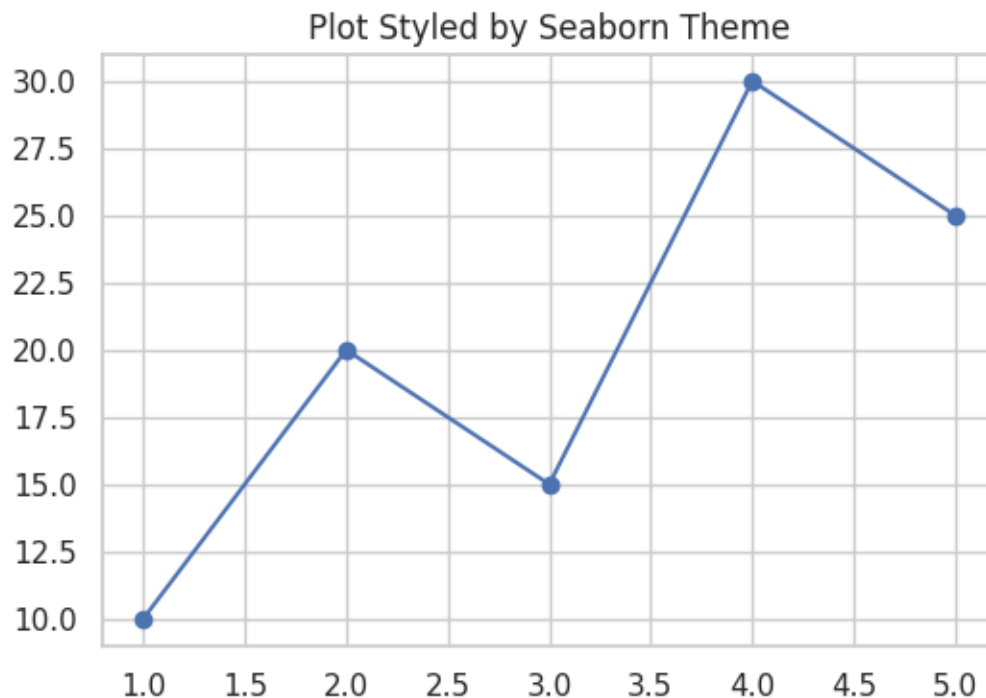
## CORE STRUCTURES: THEMES & THE TIPS DATASET

### 0.1.3 1. Setting a Theme

Seaborn's biggest advantage is its built-in, attractive default styles. Calling `sns.set_theme()` instantly changes the appearance of grids, fonts, and colors for every subsequent plot. Code snippet to apply a theme:

```
[2]: # Applying Seaborn's default theme globally
sns.set_theme(style="whitegrid", palette="deep")

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot([1, 2, 3, 4, 5], [10, 20, 15, 30, 25], marker='o')
ax.set_title("Plot Styled by Seaborn Theme")
plt.show()
```



### 0.1.4 2. Loading a Built-in Dataset

Seaborn ships with several sample datasets that are perfect for learning and demonstration. The `tips` dataset records restaurant bills, tips, and customer details. Code snippet to load and preview the dataset:

```
[3]: tips = sns.load_dataset("tips")

print("--- Tips Dataset Preview ---")
tips.head()
```

--- Tips Dataset Preview ---

```
[3]:   total_bill  tip  sex smoker  day  time  size
0      16.99  1.01 Female    No  Sun  Dinner    2
1      10.34  1.66  Male    No  Sun  Dinner    3
2      21.01  3.50  Male    No  Sun  Dinner    3
3      23.68  3.31  Male    No  Sun  Dinner    2
4      24.59  3.61 Female    No  Sun  Dinner    4
```

#

CORE GRAPH TYPES

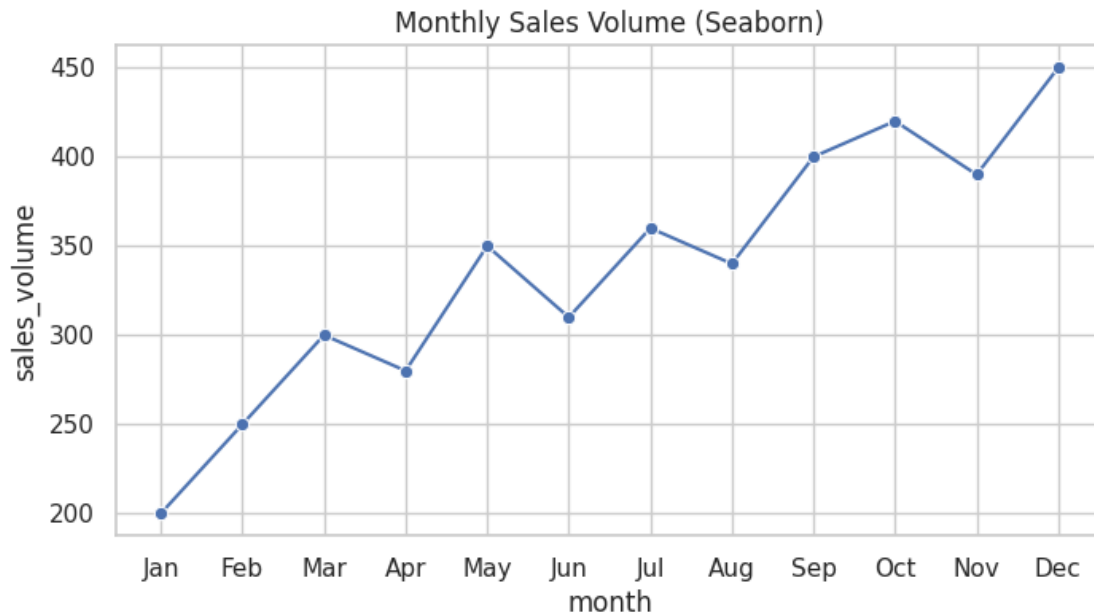
### 0.1.5 1. Line Plot

**Description:** Similar to a Matplotlib line plot, but Seaborn automatically adds confidence intervals when there are multiple observations per x-value. **Use Case:** Visualizing a trend across time with variability shown automatically. Code snippet:

```
[4]: monthly_metrics = pd.DataFrame({
    'month':_
    ↪ ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
    'sales_volume': [200, 250, 300, 280, 350, 310, 360, 340, 400, 420, 390, 450]
})

print("--- Creating a Seaborn Line Plot ---")
plt.figure(figsize=(8, 4))
sns.lineplot(data=monthly_metrics, x='month', y='sales_volume', marker='o')
plt.title("Monthly Sales Volume (Seaborn)")
plt.show()
```

--- Creating a Seaborn Line Plot ---

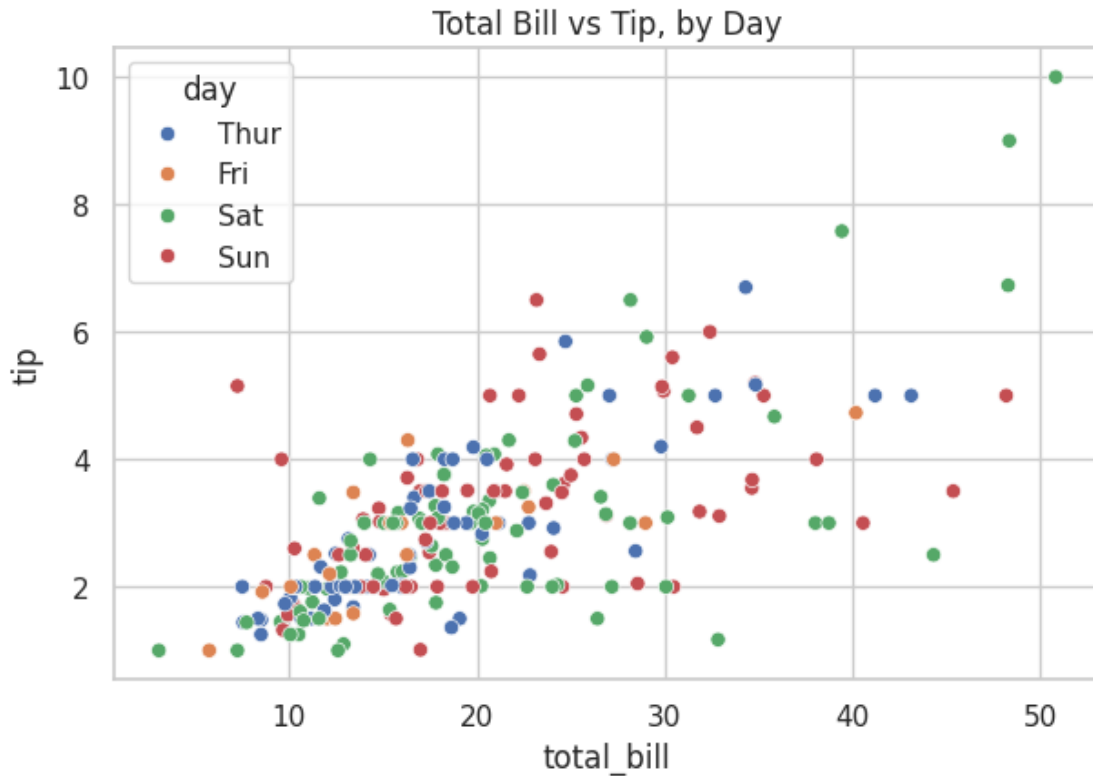


### 0.1.6 2. Scatter Plot

**Description:** Plots two numeric variables against each other, with easy options to color/size points by a third categorical variable using **hue**. **Use Case:** Exploring the relationship between total bill and tip amount, colored by day. Code snippet:

```
[5]: print("--- Creating a Seaborn Scatter Plot ---")
plt.figure(figsize=(7, 4.5))
sns.scatterplot(data=tips, x="total_bill", y="tip", hue="day")
plt.title("Total Bill vs Tip, by Day")
plt.show()
```

--- Creating a Seaborn Scatter Plot ---



### 0.1.7 3. Bar Chart

**Description:** Displays average (aggregated) values for categories, automatically adding error bars to show confidence intervals. **Use Case:** Comparing average bill totals across different days of the week. Code snippet:

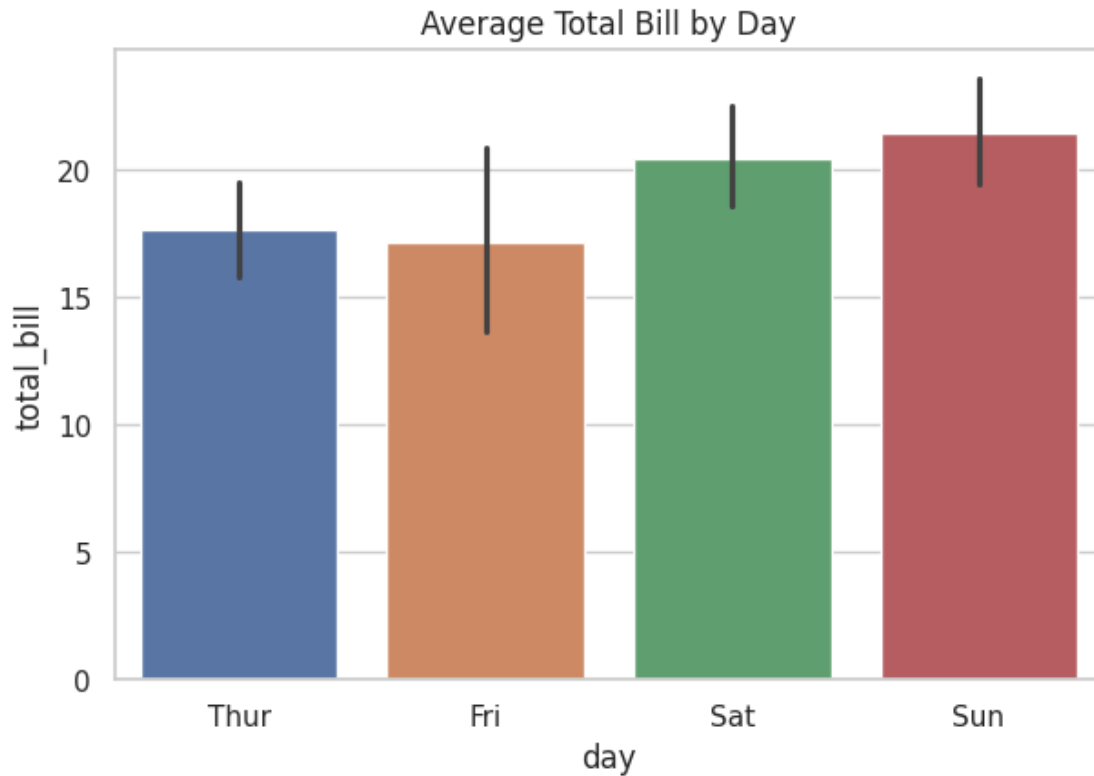
```
[6]: print("--- Creating a Seaborn Bar Chart ---")
plt.figure(figsize=(7, 4.5))
sns.barplot(data=tips, x="day", y="total_bill", palette="deep")
plt.title("Average Total Bill by Day")
plt.show()
```

--- Creating a Seaborn Bar Chart ---

/tmp/ipykernel\_570/3368610843.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=tips, x="day", y="total_bill", palette="deep")
```

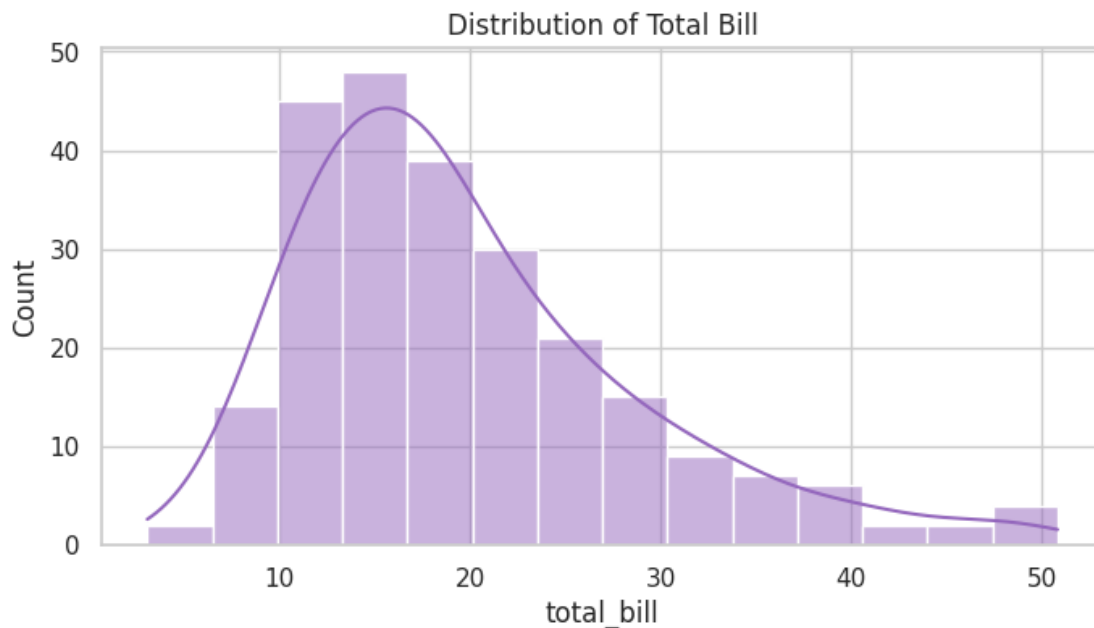


#### 0.1.8 4. Histogram

**Description:** Visualizes the distribution of a numeric variable, optionally overlaying a smooth density curve (KDE) to reveal skewness or multi-modality. **Use Case:** Analyzing how total bill amounts are distributed. Code snippet:

```
[7]: print("--- Creating a Seaborn Histogram ---")
plt.figure(figsize=(8, 4))
sns.histplot(data=tips, x="total_bill", kde=True, color='#9467bd')
plt.title("Distribution of Total Bill")
plt.show()
```

--- Creating a Seaborn Histogram ---



### 0.1.9 5. Box Plot

**Description:** Summarizes a distribution using quartiles, median, and outliers — useful for comparing groups. **Use Case:** Comparing bill totals across days, including outlier detection. Code snippet:

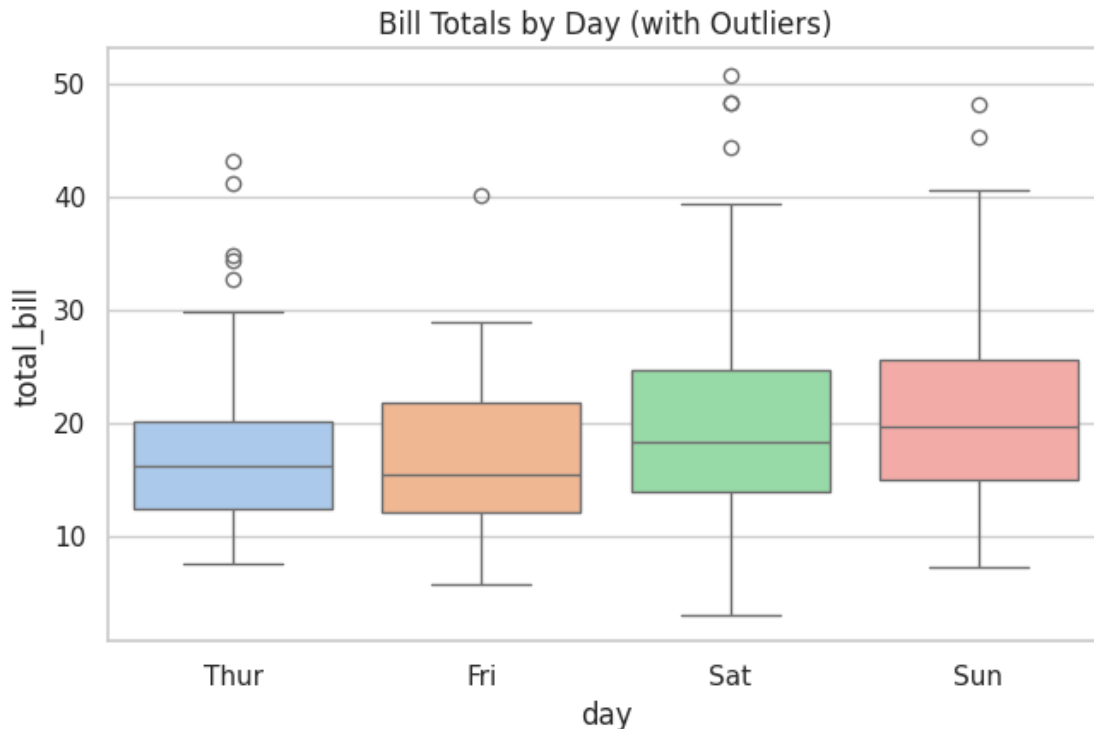
```
[8]: print("--- Creating a Seaborn Box Plot ---")
plt.figure(figsize=(7.5, 4.5))
sns.boxplot(data=tips, x="day", y="total_bill", palette="pastel")
plt.title("Bill Totals by Day (with Outliers)")
plt.show()
```

--- Creating a Seaborn Box Plot ---

/tmp/ipykernel\_570/2682877859.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=tips, x="day", y="total_bill", palette="pastel")
```



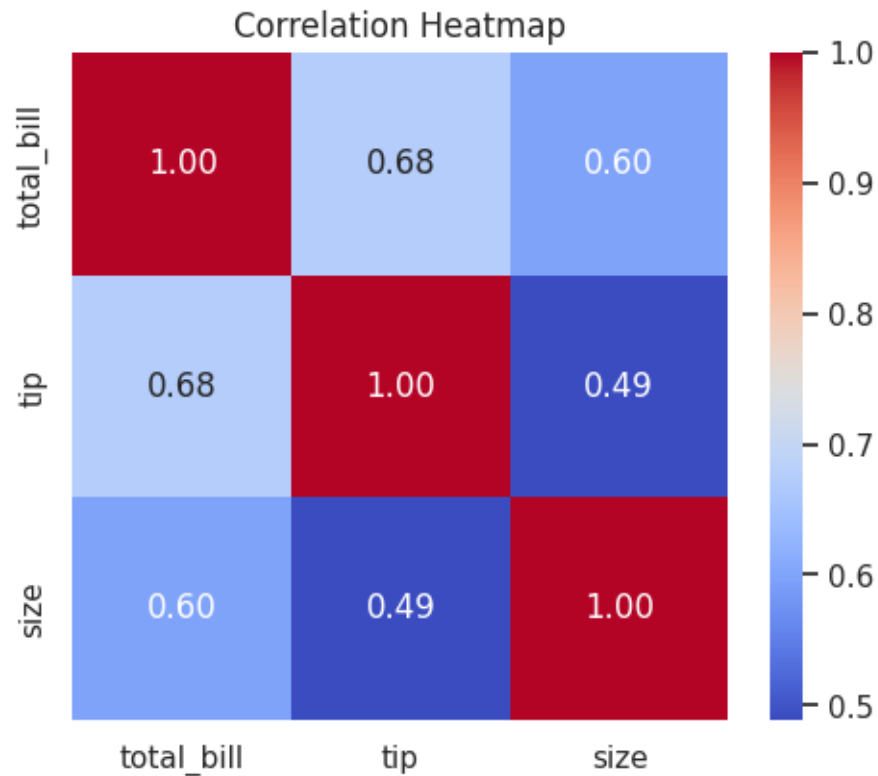
### 0.1.10 6. Heatmap

**Description:** Displays matrix-style data as a color-coded grid — perfect for visualizing correlations between multiple numeric columns at once. **Use Case:** Quickly spotting which variables are strongly correlated with each other. Code snippet:

```
[9]: print("--- Creating a Seaborn Heatmap ---")
numeric_cols = tips[['total_bill', 'tip', 'size']]
corr_matrix = numeric_cols.corr()

plt.figure(figsize=(5.5, 4.5))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap")
plt.show()
```

--- Creating a Seaborn Heatmap ---



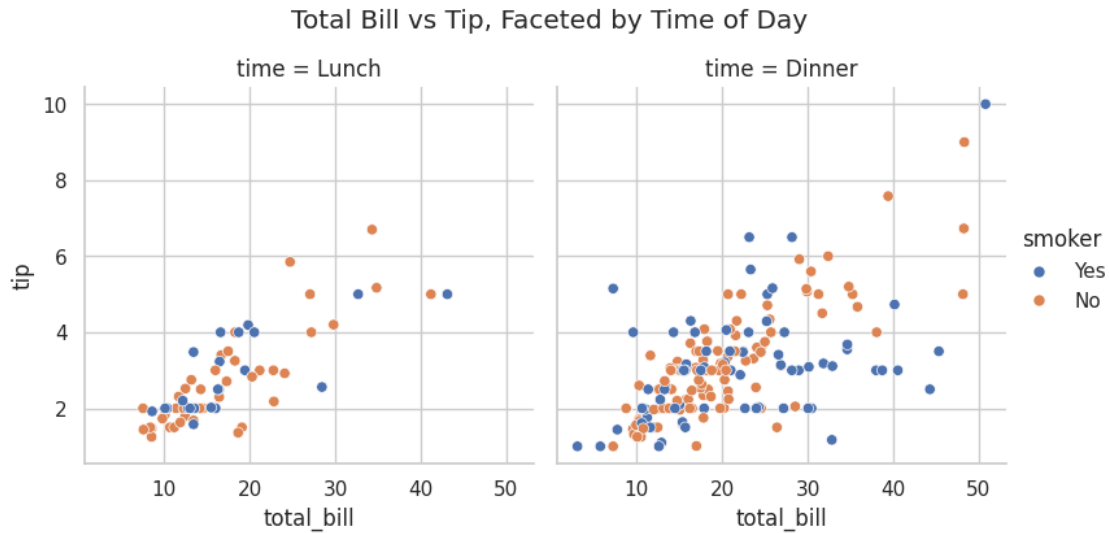
#

#### ADVANCED FEATURE: FACETING WITH FIGURE-LEVEL PLOTS

Unlike Matplotlib, Seaborn can automatically create an entire grid of subplots split by a categorical variable, using a single line of code via `sns.relplot`, `sns.catplot`, or `sns.displot`. Code snippet to facet scatter plots by time of day:

```
[10]: print("--- Creating a Faceted Seaborn Plot ---")
g = sns.relplot(data=tips, x="total_bill", y="tip", col="time", hue="smoker",
               height=4, aspect=1)
g.fig.suptitle("Total Bill vs Tip, Faceted by Time of Day", y=1.05)
plt.show()
```

--- Creating a Faceted Seaborn Plot ---



#

#### ADVANTAGES OF SEABORN

- **Beautiful Defaults:** Attractive themes and color palettes with zero manual styling required.
- **Statistical Awareness:** Automatically computes and displays confidence intervals, regression lines, and aggregated statistics.
- **DataFrame Integration:** Plots directly from column names, reducing boilerplate code significantly.
- **Faceting Made Easy:** Splitting a chart into multiple subplots by category takes a single line of code.

#

#### Conclusion

In short, Seaborn is the go-to library for fast, attractive, and statistically meaningful visualizations in Python. Instead of manually configuring colors, legends, and confidence intervals in raw Matplotlib, Seaborn handles all of this automatically from simple DataFrame column references. It acts like a supercharged styling and statistics layer on top of Matplotlib, allowing you to instantly explore distributions, relationships, and categorical comparisons with minimal code.